DALHOUSIE UNIVERSITY

CSCI 5408: Data Management, Warehousing and Analytics

Assignment: Lab - 06

Banner ID: B01026328

Name: Parva Mineshbhai Patel

Date: 2025 – 02 – 16

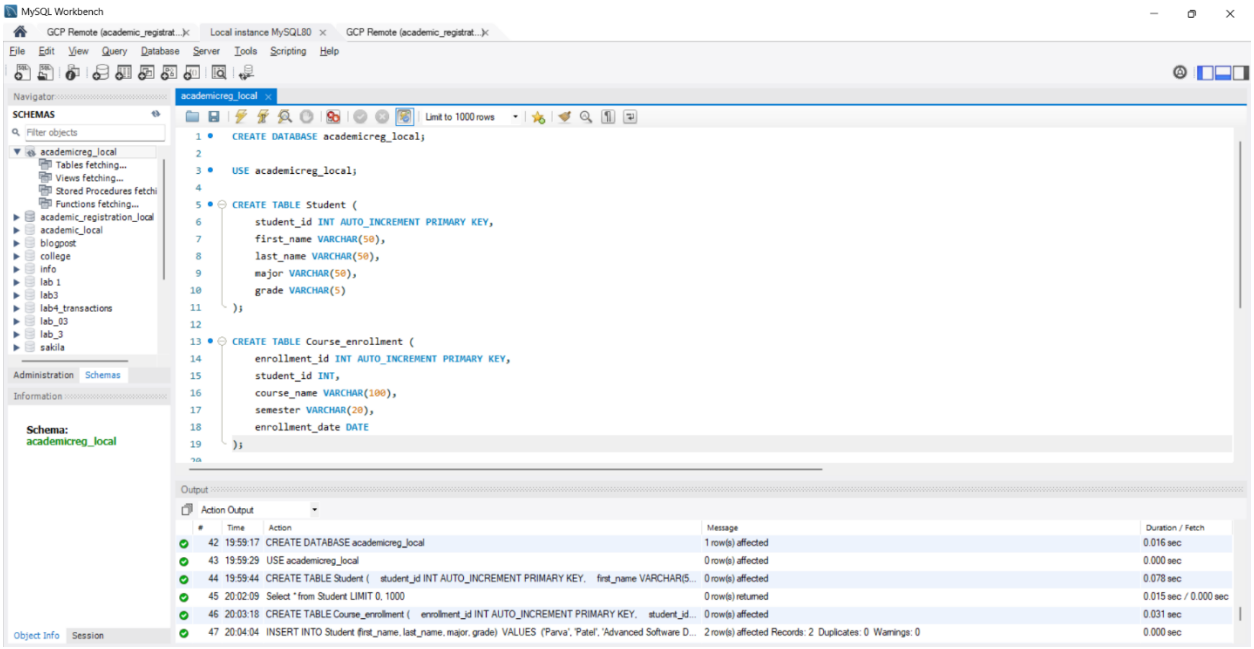
GitLab Assignment Link: https://git.cs.dal.ca/parva/
CSCI5408_W25_B01026328_ParvaMineshbhai_Patel

Contents

1) DDL commands and screenshots of the database and tables created

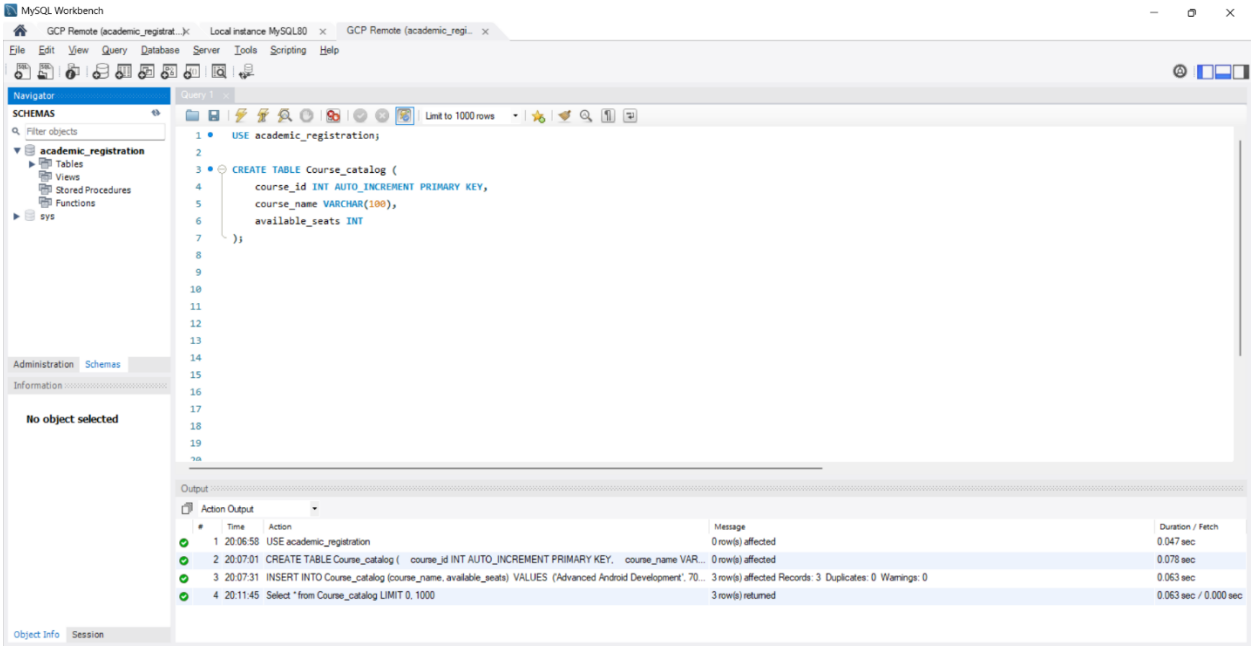
Local DB DDL commands

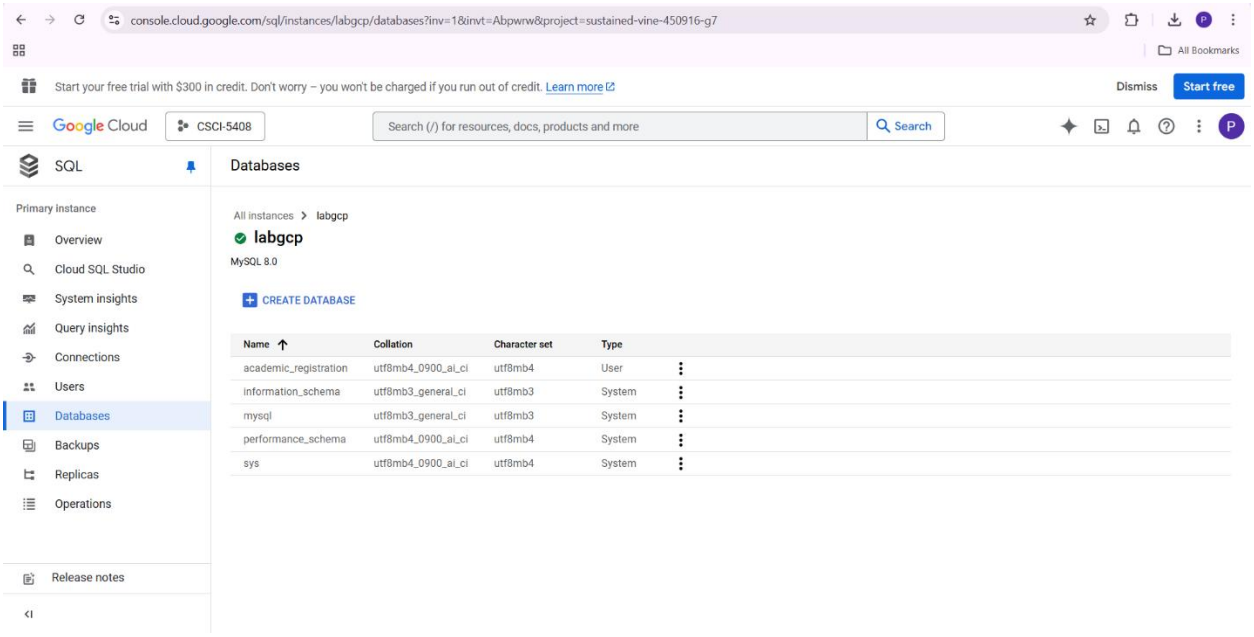
```
CREATE DATABASE academicreg_local;
USE academicreg_local;
CREATE TABLE Student (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    major VARCHAR(50),
    grade VARCHAR(5)
);
CREATE TABLE Course_enrollment (
    enrollment_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT,
    course_name VARCHAR(100),
    semester VARCHAR(20),
    enrollment_date DATE
);
```



GCP DB DDL commands

```
USE academic_registration;

CREATE TABLE Course_catalog (
    course_id INT AUTO_INCREMENT PRIMARY KEY,
    course_name VARCHAR(100),
    available_seats INT
);
```

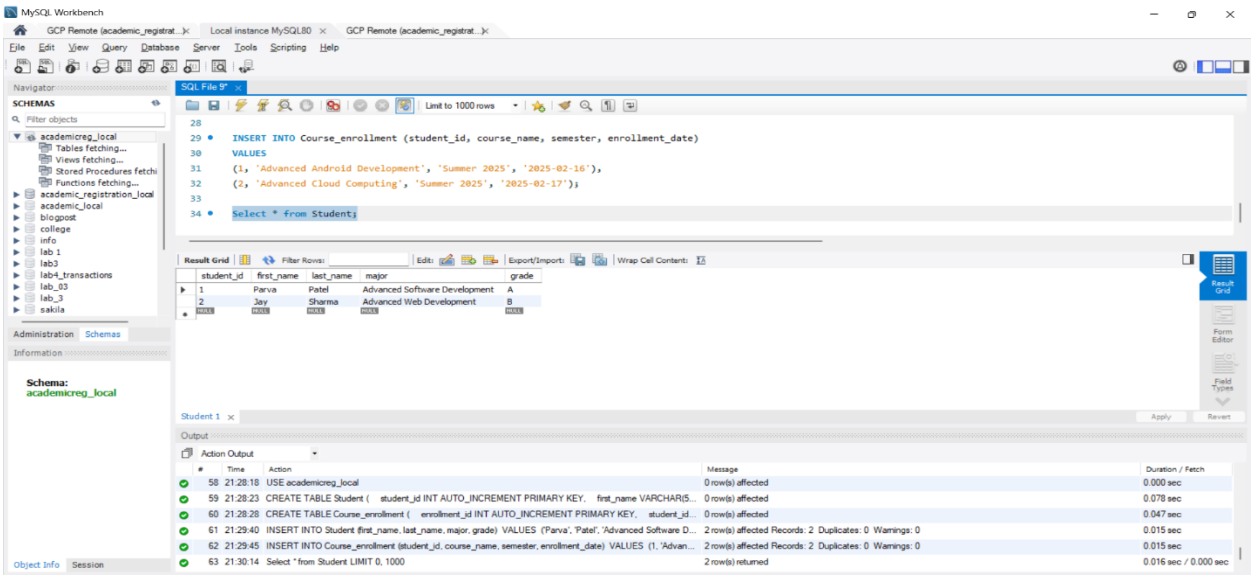


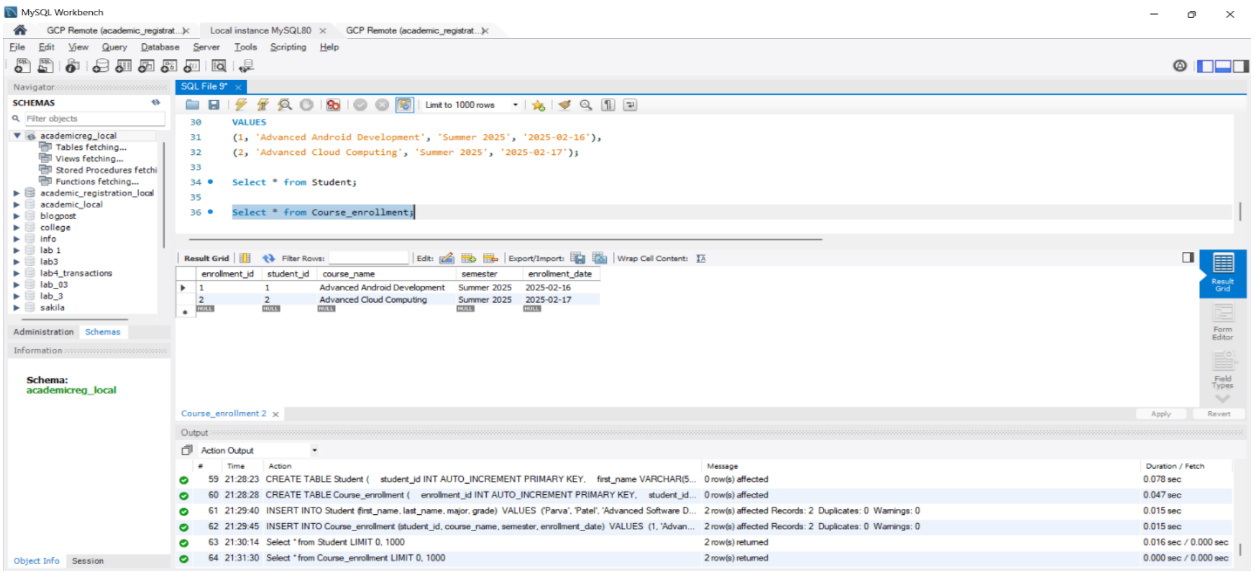


2) DML commands and screenshots of inserted dummy data

Local DB DML commands

```
USE academicreg_local;
INSERT INTO Student (first_name, last_name, major, grade)
VALUES
('Parva', 'Patel', 'Advanced Software Development', 'A'),
('Jay', 'Sharma', 'Advanced Web Development', 'B');

INSERT INTO Course_enrollment (student_id, course_name, semester, enrollment_date)
VALUES
(1, 'Advanced Android Development', 'Summer 2025', '2025-02-16'),
(2, 'Advanced Cloud Computing', 'Summer 2025', '2025-02-17');

Select * from Student;
Select * from Course_enrollment;
```

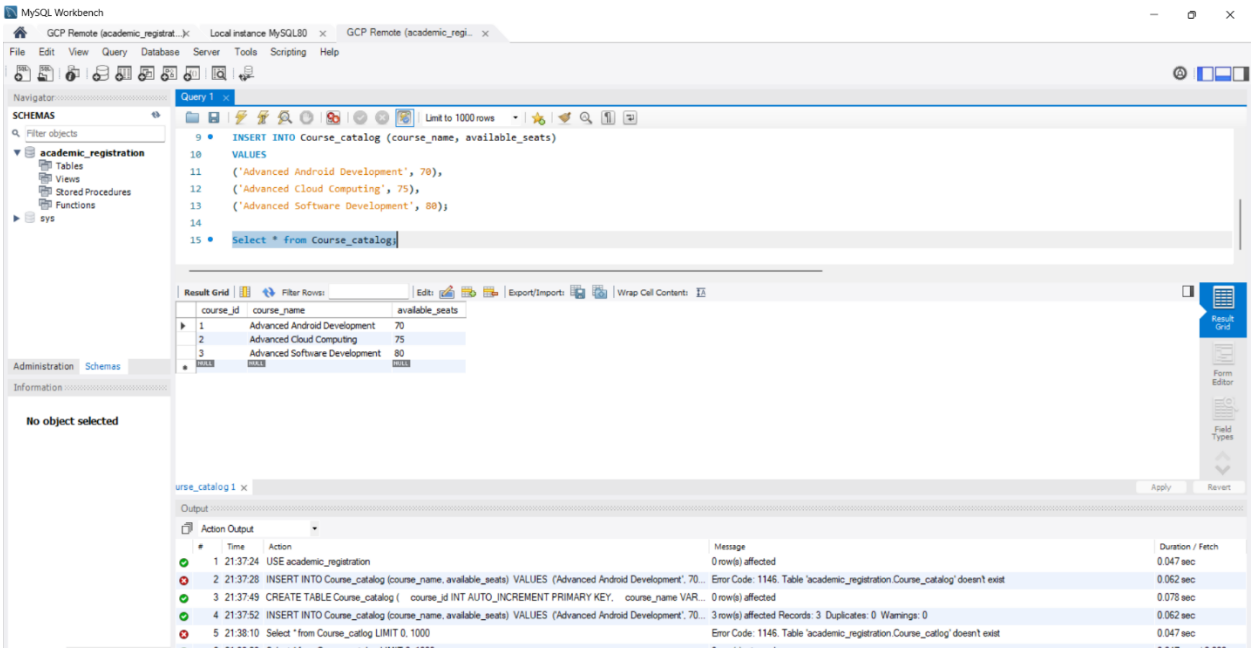




GCP DB DML commands

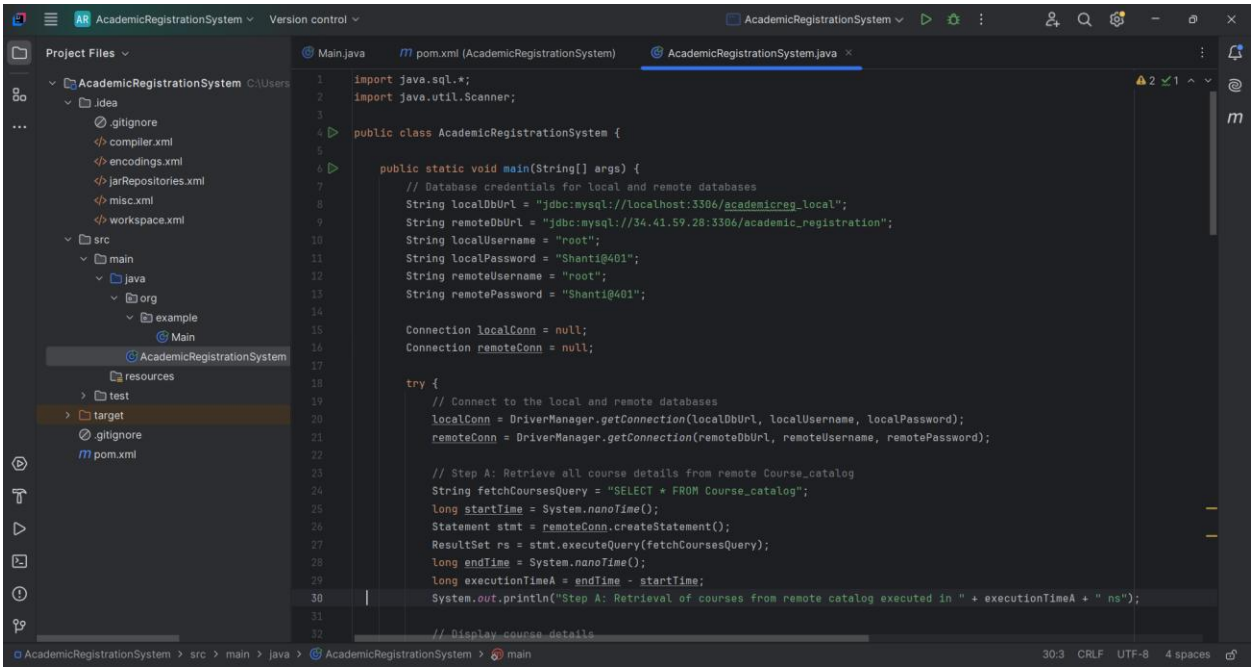
INSERT INTO Course_catalog (course_name, available_seats)
VALUES
('Advanced Android Development', 70),
('Advanced Cloud Computing', 75),
('Advanced Software Development', 80);

Select * from Course_catalog;

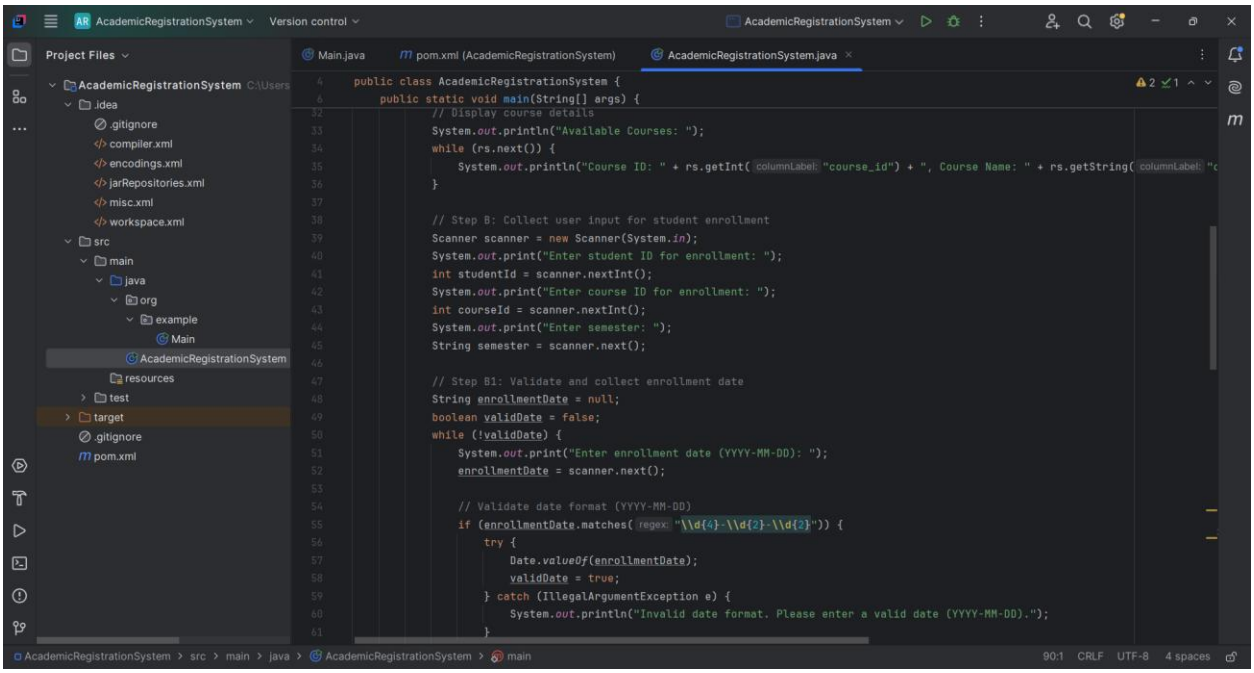


3) Java code, Program Flow, Components and Screenshots

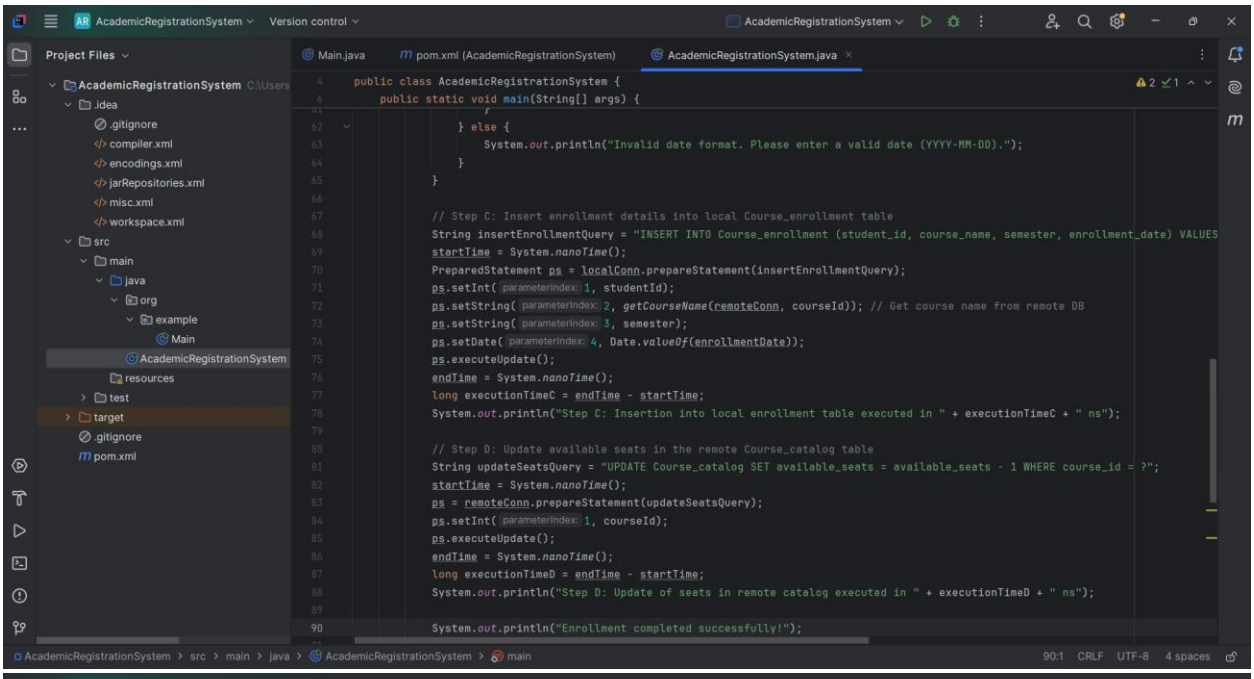
Java Program Code



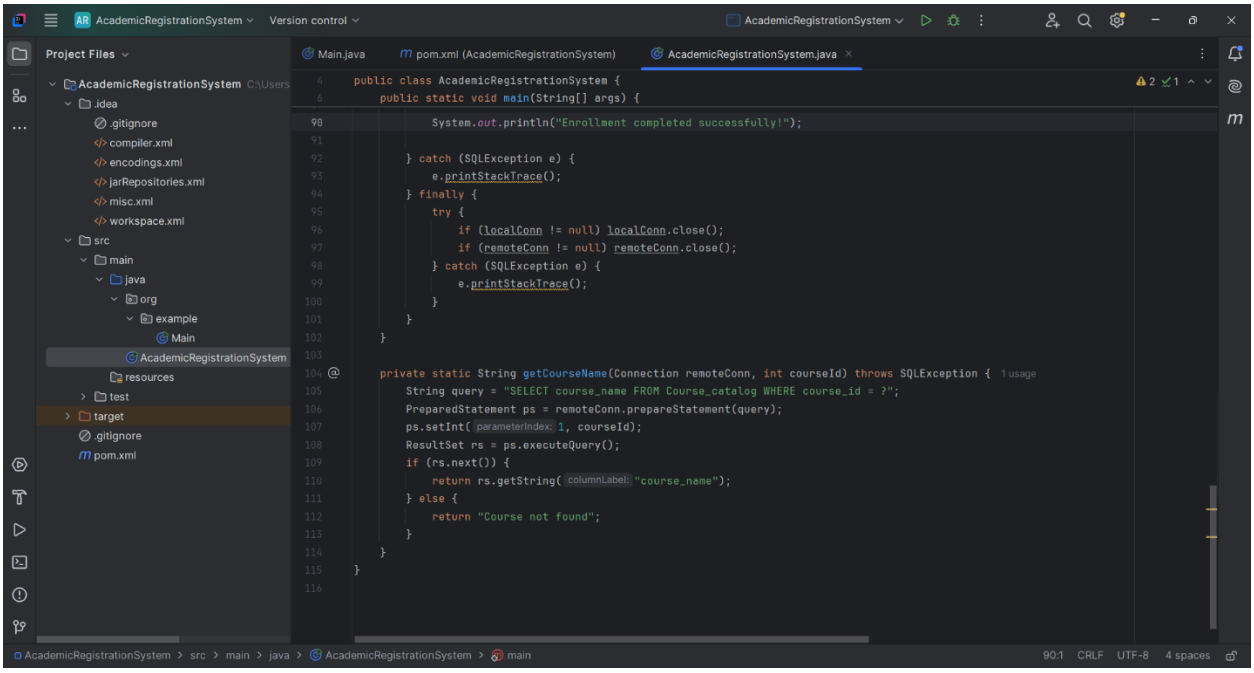
```
import java.sql.*;
import java.util.Scanner;

public class AcademicRegistrationSystem {

    public static void main(String[] args) {
        // Database credentials for local and remote databases
        String localDbUrl = "jdbc:mysql://localhost:3306/academicreg_local";
        String remoteDbUrl = "jdbc:mysql://34.41.59.28:3306/academic_registration";
        String localUsername = "root";
        String localPassword = "Shanti@401";
        String remoteUsername = "root";
        String remotePassword = "Shanti@401";

        Connection localConn = null;
        Connection remoteConn = null;

        try {
            // Connect to the local and remote databases
            localConn = DriverManager.getConnection(localDbUrl, localUsername, localPassword);
            remoteConn = DriverManager.getConnection(remoteDbUrl, remoteUsername, remotePassword);

            // Step A: Retrieve all course details from remote Course_catalog
            String fetchCoursesQuery = "SELECT * FROM Course_catalog";
            long startTime = System.nanoTime();
            Statement stmt = remoteConn.createStatement();
            ResultSet rs = stmt.executeQuery(fetchCoursesQuery);
            long endTime = System.nanoTime();
            long executionTimeA = endTime - startTime;
            System.out.println("Step A: Retrieval of courses from remote catalog executed in " + executionTimeA + " ns");

            // Display course details
```



```
public class AcademicRegistrationSystem {
    public static void main(String[] args) {
            // Display course details
            System.out.println("Available Courses: ");
            while (rs.next()) {
                System.out.println("Course ID: " + rs.getInt( columnLabel: "course_id") + ", Course Name: " + rs.getString( columnLabel: "c
            }

            // Step B: Collect user input for student enrollment
            Scanner scanner = new Scanner(System.in);
            System.out.print("Enter student ID for enrollment: ");
            int studentId = scanner.nextInt();
            System.out.print("Enter course ID for enrollment: ");
            int courseId = scanner.nextInt();
            System.out.print("Enter semester: ");
            String semester = scanner.next();

            // Step B1: Validate and collect enrollment date
            String enrollmentDate = null;
            boolean validDate = false;
            while (!validDate) {
                System.out.print("Enter enrollment date (YYYY-MM-DD): ");
                enrollmentDate = scanner.next();

                // Validate date format (YYYY-MM-DD)
                if (enrollmentDate.matches( regex: "\\d{4}-\\d{2}-\\d{2}")) {
                    try {
                        Date.valueOf(enrollmentDate);
                        validDate = true;
                    } catch (IllegalArgumentException e) {
                        System.out.println("Invalid date format. Please enter a valid date (YYYY-MM-DD).");
                    }
```



```
public class AcademicRegistrationSystem {
    public static void main(String[] args) {
                } else {
                    System.out.println("Invalid date format. Please enter a valid date (YYYY-MM-DD).");
                }
            }

            // Step C: Insert enrollment details into local Course_enrollment table
            String insertEnrollmentQuery = "INSERT INTO Course_enrollment (student_id, course_name, semester, enrollment_date) VALUES
            startTime = System.nanoTime();
            PreparedStatement ps = localConn.prepareStatement(insertEnrollmentQuery);
            ps.setInt( parameterIndex: 1, studentId);
            ps.setString( parameterIndex: 2, getCourseName(remoteConn, courseId)); // Get course name from remote DB
            ps.setString( parameterIndex: 3, semester);
            ps.setDate( parameterIndex: 4, Date.valueOf(enrollmentDate));
            ps.executeUpdate();
            endTime = System.nanoTime();
            long executionTimeC = endTime - startTime;
            System.out.println("Step C: Insertion into local enrollment table executed in " + executionTimeC + " ns");

            // Step D: Update available seats in the remote Course_catalog table
            String updateSeatsQuery = "UPDATE Course_catalog SET available_seats = available_seats - 1 WHERE course_id = ?";
            startTime = System.nanoTime();
            ps = remoteConn.prepareStatement(updateSeatsQuery);
            ps.setInt( parameterIndex: 1, courseId);
            ps.executeUpdate();
            endTime = System.nanoTime();
            long executionTimeD = endTime - startTime;
            System.out.println("Step D: Update of seats in remote catalog executed in " + executionTimeD + " ns");

            System.out.println("Enrollment completed successfully!");
```



```
            System.out.println("Enrollment completed successfully!");

        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            try {
                if (localConn != null) localConn.close();
                if (remoteConn != null) remoteConn.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }

    private static String getCourseName(Connection remoteConn, int courseId) throws SQLException {  1 usage
        String query = "SELECT course_name FROM Course_catalog WHERE course_id = ?";
        PreparedStatement ps = remoteConn.prepareStatement(query);
        ps.setInt( parameterIndex: 1, courseId);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return rs.getString( columnLabel: "course_name");
        } else {
            return "Course not found";
        }
    }
}
```

Program Flow Explanation:

1. **Database Connection**:
    - The program establishes connections to both **local** and **remote databases** using JDBC.
    - **localConn** is for the local database (stores student enrollments), and **remoteConn** is for the remote database (stores course catalog and available seats).
2. **Step A: Retrieve Course Details**:
    - A query **SELECT * FROM Course_catalog** retrieves the available courses from the remote database.

- The courses are displayed, showing their **course_id, course_name, and available_seats**.

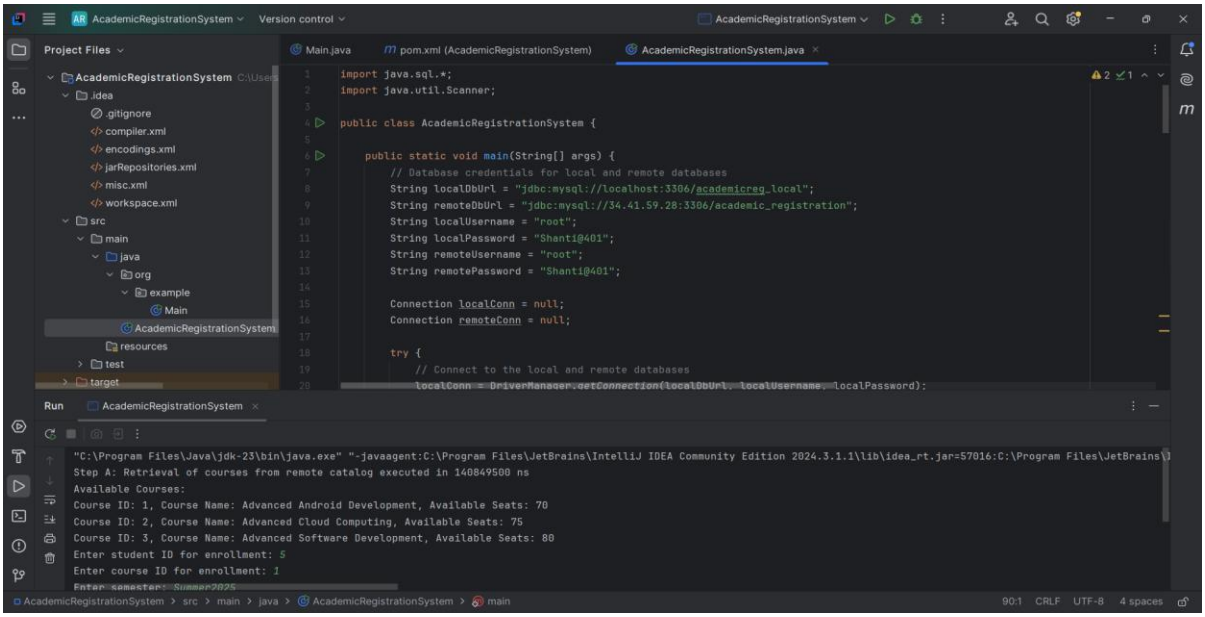
3. **Step B: User Input**:
    - The program collects user input for enrollment:
        - **Student ID**
        - **Course ID** (to enroll in)
        - **Semester**
    - It then validates the **enrollment date** format (YYYY-MM-DD). If the date format is invalid, the program prompts the user to enter it again.
4. **Step C: Insert Enrollment Details**:
    - The program inserts the student enrollment data into the **local database** (Course_enrollment table) with the details provided.
    - The course name is fetched from the remote database using **getCourseName()** to match the **course ID** provided by the user.
5. **Step D: Update Available Seats**:
    - After the enrollment is successfully recorded, the program updates the **available_seats** in the remote database (Course_catalog table) by decreasing it by 1 for the enrolled course.
6. **Execution Time**:
    - The program calculates and prints the execution time for each step (A, C, and D) in nanoseconds to measure performance.
7. **Closing Connections**:
    - After all operations are completed, database connections are closed to avoid memory leaks or any open connections.

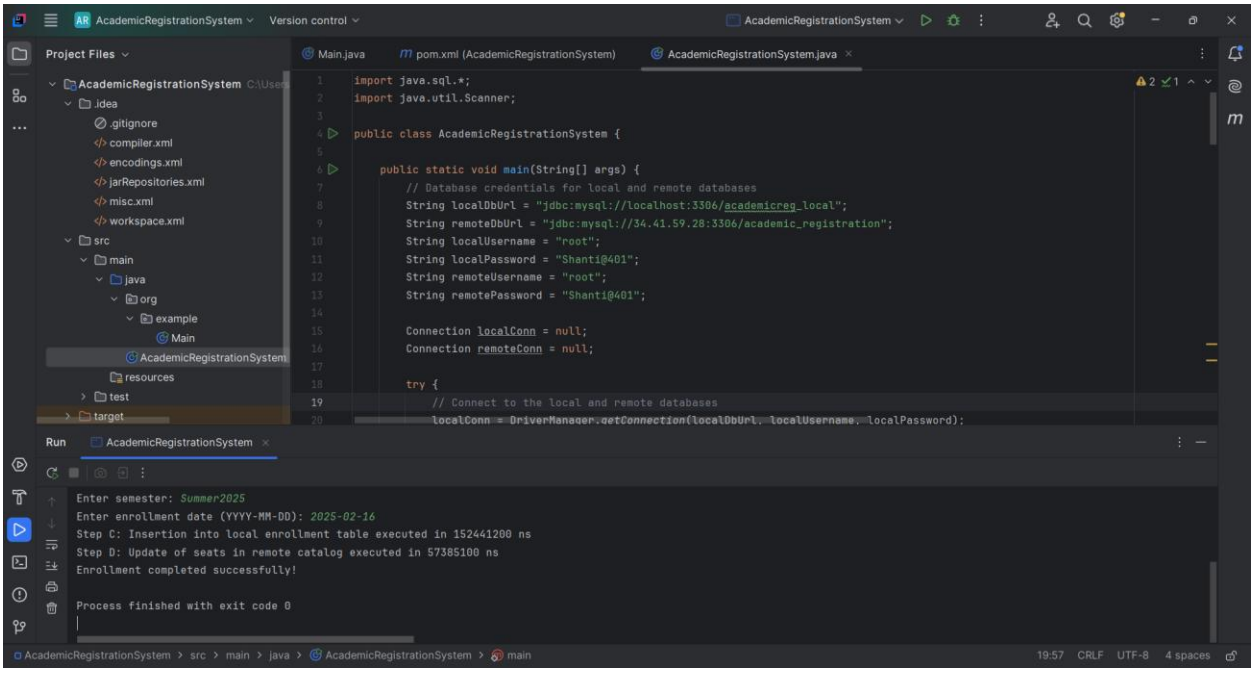
Component Documentation:

- **Database Connections**:
    - **localConn**: Connection to the local MySQL database for student enrollments.
    - **remoteConn**: Connection to the remote GCP MySQL database for courses and available seats.
- **SQL Queries**:
    - **Fetch Courses**: Retrieves available courses from the remote database.
    - **Insert Enrollment**: Adds student enrollment data to the local database.
    - **Update Seats**: Decreases available seats in the remote course catalog.
- **Helper Methods**:
    - **getCourseName()**: Fetches course name from the remote database based on **course_id.**
- **Performance Measurement**:
    - **Execution Time**: Measures and prints the execution time of each SQL query step.

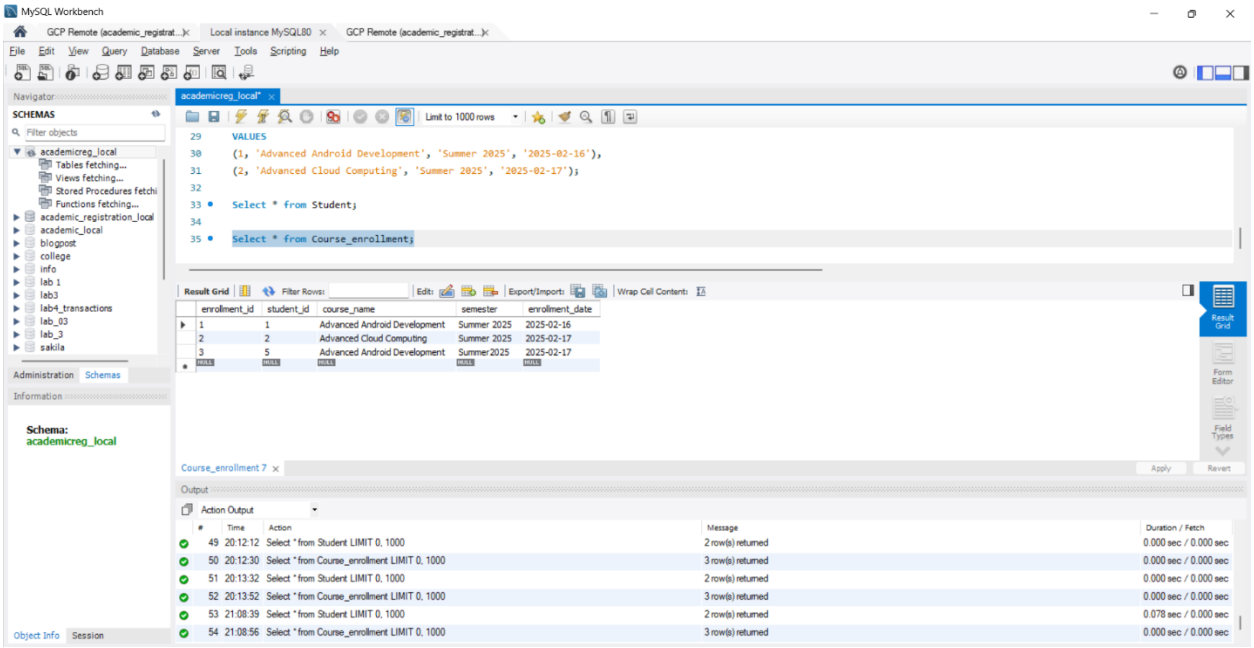
ScreenShots

Execution of Java Code

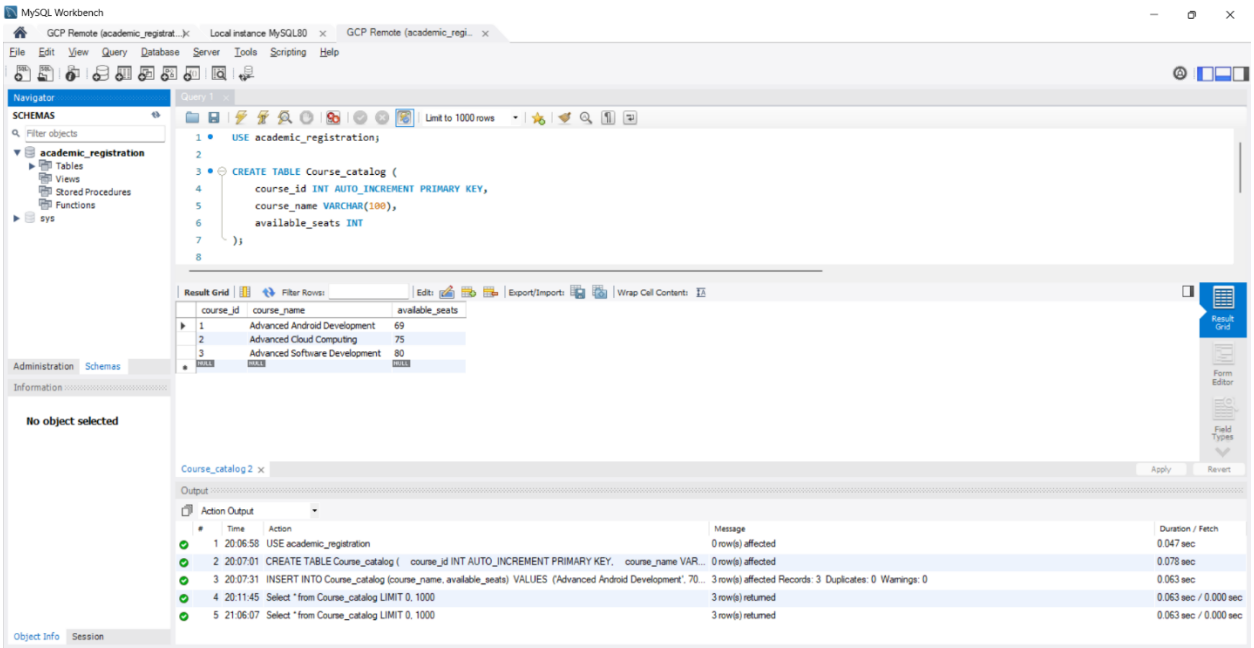




Course_enrollment table in local db after running java code



Course_Catalog table in MYSQL after running java code (GCP Remote)



4) Explanation of execution time differences between local and remote database operations

1. **Local Database Operations:**

    - Operations like **Step C** inserting enrollment data into the local **Course_enrollment** table are faster because they occur on the local machine. There's no network latency as the database is running locally.
    - The query is executed directly on the local MySQL database, which leads to quick processing times.

2. **Remote Database Operations:**

- **Step A** retrieving course details from the remote **Course_catalog** table and **Step D** updating available seats in the remote database are slower due to the need to communicate with the remote MySQL server hosted on Google Cloud Platform (GCP).
- Each query involves data being sent over the internet, leading to added delays due to network traffic, security checks, encryption, and server-side load balancing.
- Fetching data or performing updates over a remote connection requires round-trip communication, making it inherently slower than operations on the local machine.

5) References

1. MySQL Workbench [Software]. Oracle Corporation. Available: https://www.mysql.com/products/workbench/ [Accessed: February 16, 2025].
2. Google Cloud Platform (GCP) [Online]. Google. Available: https://cloud.google.com/ [Accessed: February 16, 2025].
3. MySQL Connector/J (JDBC Driver) [Software]. Oracle Corporation. Available: https://dev.mysql.com/downloads/connector/j/ [Accessed: February 16, 2025].